The Journal of Open Source Software

Numpydantic: Static and dynamic array constraint validation for data standards and working programmers

Jonny L. Saunders [1,2] **and Daniel Aharoni** [1]

1 University of California, Los Angeles. Department of Neurology. **2** Peertech Global Cyberindustrial Concern LLC, LLC.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Johan Larsson

Reviewers:

- @niksirbi
- @kirangadhave
- @JoeZiminski

Submitted: 11 June 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Numpydantic is a minimalistic Python package that provides abstract types and a syntax for specifying constrained n-dimensional arrays. Its constraints can be validated both dynamically at runtime and statically using mypy. Rather than being bound to a single array framework, numpydantic is a generic array specification with an extensible interface system that currently supports NumPy, HDF5, zarr, and video files. Designed to expose arrays to `Pydantic`'s validation framework, its `NDArray` type allows researchers to compose multiple arrays into validating data models, including generation of JSON schema, roundtrip serialization, and lazy proxies for on-disk arrays. A mypy plugin extends the quality of life improvements from static type analysis to the world of array programming. Numpydantic is part of a broader effort to lower barriers to structured data and provide a path to true cross-standard interoperability: decoupling schemas, implementations, and serialization formats for formal data standards, and providing simple tools responsive to the needs of everyday research programming.

Statement of need

Arrays are rarely formless bundles of numbers, they are "videos," "time series," "training and testing data," etc. where attributes like orientation, shape, and dtype are an intrinsic part of their definition as a type. Accordingly, outside of array frameworks themselves, array code rarely accepts arbitrary arrays. In Python this information lives only in docstrings, is invisible to type checkers, and requires large amounts of repetitive boilerplate to validate.

The lofty ambitions of open data require standards and formats (Wilkinson et al., 2016). Scientific data standards must specify array constraints, but the implementation of those constraints often binds the abstract schema to a single array framework and serialization format. Researchers have a rich set of static analysis tools to assist with programming, but still struggle with the hundreds of intermediate arrays that flow through their code.

This tension creates two unhappy local minima: on one side, data standards are often brittle and difficult to integrate into the code used for real-world analysis, relegated to a conversion stage after the work is done that can be lossy and labor-intensive. On the other, labs run on ad-hoc, local data structures that make even intra-lab collaboration difficult.

In the middle is a gap in tooling that responds to both categories of needs:

- Strict, validating, metadata enriched schemas with loose coupling to storage formats and programming interfaces
- Simple, low-overhead structure that can embed in existing code and support researcher-developer experience.

DRAFT

State of the field

Python's type annotation system has evolved dramatically since its introduction in 2014 (Python Software Foundation, n.d.; van Rossum et al., 2014). Typing arrays is an ongoing challenge, motivating type system features like variadic generics for shape specification (Mendoza et al., 2020). NumPy (Harris et al., 2020) introduced type annotations for its arrays in 2021 (*NumPy 1.20.0 Release Notes*, 2021), with steady improvements over time[1]. NumPy's types allow specifying shape and dtype, but not extended expressions like explicit dimension sizes, size ranges, etc. Numpydantic extends previous work on runtime-checkable types, initially vendoring `nptyping` (Hagenaars, 2019)[2] and extending and updating its syntax, and later replacing it with correct contemporary type annotations in v2.0.0.

Other array frameworks like Dask (Dask Development Team, 2016) and Zarr (Miles et al., 2020) currently allow typing only for dtype or a custom metadata object.

Dataclasses (Smith, 2017) use type annotations to create intelligible data structures, but the annotations are unvalidated and used solely in static analysis. Pydantic (Colvin et al., 2026) is the most widely used library in Python for declaring runtime-validating dataclass-like models, but it lacks builtin support for arrays.

The closest sibling to numpydantic is likely pandera (Bantilan, 2020), which provides schemas for dataframes across multiple backends (and, recently, complex multi-arrays from xarray). Second would be jaxtyping (Kidger, 2022), which provides custom string-based constraints on shapes (including cross-array constraints) enforceable with `typeguard` (Grönholm, 2015) and at runtime for JAX, PyTorch, NumPy, MLX, and TensorFlow arrays and tensors. Third is pydantic-numpy (Tartanoglu & Heindl, 2022) which more narrowly supports NumPy arrays for Pydantic. We are not aware of other projects that provide array specifications and validation across multiple array backends with a single annotation consistent with the emerging consensus on array typing[3].

Software design

Numpydantic is designed to be a dependency you don't have to think about - it has two required dependencies, NumPy and Pydantic, which are already likely present in any package that wants to use it. It can be used with or without Pydantic[4]: its annotations can be used in the typing layer without modifying existing runtime code.

Support for multiple array frameworks is implemented as a set of interface classes, where each interface overrides the methods needed to match input to an interface, extract shape and dtype, serialize and deserialize JSON, among other features.

[1] See also the predecessor `numpy-stubs` (numpy, 2017) and the experimental numtype (numpy, 2025).
[2] As of writing, `nptyping` is unmaintained, without update since February 2023.
[3] If you maintain an array spec library for Python, please contact us and we will link to you from our docs!
[4] Pydantic is a required dependency, but the annotations can be used outside of Pydantic models

DRAFT

Pydantic Models

At its simplest, numpydantic does this:

```python
import numpy as np
from pydantic import BaseModel
from numpydantic import NDArray, Range, Shape, L

class MyModel(BaseModel):
    array: NDArray[Shape[int, int, int, L[3]], np.uint8]

# numpy
MyModel(array=np.zeros((100, 1080, 1920, 3), dtype=np.uint8))
# dask
MyModel(array=dask.array.zeros((100, 1080, 1920, 3), dtype=np.uint8))
# hdf5
MyModel(array=('data.h5', '/nested/dataset'))
# zarr
MyModel(array=zarr.zeros((100, 1080, 1920, 3), dtype=np.uint8))
MyModel(array=('data.zarr', '/nested/dataset'))
# video
MyModel(array="data.mp4")
```

When used in Pydantic models, numpydantic provides runtime validation for array specifications, as well as access to on-disk arrays by reference.

By default, an array from any supported array backend can be used (as long as it matches the specification), but an explicit array backend can be specified using an annotated type

```python
class MyModel(BaseModel):
    array: Annotated[np.ndarray, NDArraySchema(Shape(1, 2, 3), np.uint8)]
```

Numpydantic constructs a valid JSON schema for its annotations, making them portable across languages, and usable to accept array data via web APIs via, e.g. FastAPI.

This simple example:

```python
class MyModel(BaseModel):
    array: NDArray[Shape[L[2], Range[L[2], L[5]]], np.float16]
MyModel.model_json_schema()
```

produces JSON Schema[5]:

```
{ "properties": { "array": {
  "dtype": "numpy.float16",
  "items": {
    "items": {
      "minimum": -65504.0, "maximum": 65504.0,
      "type": "number"
    },
    "minItems": 2, "maxItems": 5,
    "type": "array"
  },
  "minItems": 2, "maxItems": 2,
  "type": "array"
}}}
```

[5] All examples are abbreviated and formatted for clarity.

DRAFT

Numpydantic supports roundtripping array data to and from JSON, including any metadata needed to reconstruct an array, with on-disk arrays optionally dumped as a reference rather than loading and serializing the whole array.

e.g. for a dataset with scalar metadata alongside array data split between in-memory and on-disk arrays, as would be common in both data standards and unstructured lab code:

```
MyDataset(
  name   = "My Dataset",
  time   = datetime.now(UTC),
  scores = np.zeros((2, 2)),
  raw    = ZarrArrayPath("./data.zarr", "/dataset"),
  video  = Path('./video.mp4')
).model_dump_json(round_trip=True)
```

A lightweight metadata descriptor can be dumped to JSON and reloaded later:

```
{ "name": "My Dataset",
  "time": "2026-06-11T02:17:53.973174Z",
  "scores": {
    "type": "numpy", "dtype": "float64", "shape": [ 2, 2 ],
    "value": [ [ 0.0, 0.0 ], [ 0.0, 0.0 ] ]
  },
  "raw": {
    "type": "zarr",      "dtype": "float64",
    "file": "data.zarr", "path": "/dataset",
    "info": {
      "shape": [2000, 2000], "chunk_grid": "..."
    }
  },
  "video": { "type": "video", "file": "video.mp4" }
}
```

Standalone Use

Numpydantic can be used without Pydantic to annotate normal functions and classes!

```
def some_analysis(array: NDArray[Shape[L[1], L[2], L[3]]]) -> NDArray[Shape[L[2], L[4],
    ...
```

These annotations can also be validated at runtime without Pydantic using a tool like beartype (Curry, 2020).

NDArray can also be used as a runtime type for instance checking and ad-hoc validation

```
>>> Grayscale = NDArray[Shape[int, int], np.uint8]
>>> array = np.zeros((100, 100), np.uint8)
>>> isinstance(np.zeros((100, 100), np.uint8), Grayscale)
True
>>> isinstance(np.zeros((100,)), Grayscale)
False
>>> Grayscale(np.zeros((100,), np.uint8))
ShapeError: Invalid shape! expected shape [int, int], got shape (100,)
>>> Grayscale(np.zeros((100, 100), np.float64))
DtypeError: Invalid dtype! expected <class 'numpy.uint8'>, got float64
```

DRAFT

Mypy plugin

A mypy plugin supports static analysis of numpydantic's specification syntax, enriching array constructors with shape and dtype information for any supported array backend.

```
reveal_type(np.zeros((3, 4), dtype=np.uint8))
reveal_type(zarr.zeros((3, 4), dtype=np.uint8))
```

Without the plugin:

```
Revealed type is "numpy.ndarray[tuple[int, int], numpy.uint8]"
Revealed type is "Any"
```

With the plugin:

```
Revealed type is "numpy.ndarray[tuple[Literal[3], Literal[4]], numpy.uint8]"
Revealed type is "numpy.ndarray[tuple[Literal[3], Literal[4]], numpy.uint8]"
```

By annotating analysis and i/o functions with type information, researchers can avoid tedious bugs from misshaped or mistyped arrays, and use the runtime types for reusable, minimal boilerplate validation.

```
def analyze_grayscale(frame: NDArray[Shape[L[1080], L[1920]], np.uint8]):
    # ...

analyze_grayscale(np.zeros((1080, 1920, 3), dtype=np.uint8))
```

```
error: Argument "frame" to "analyze_grayscale" has incompatible type
  "ndarray[tuple[Literal[1080], Literal[1920], Literal[3]], numpy.uint8]";
  expected "ndarray[tuple[Literal[1080], Literal[1920]], np.uint8]"
```

Research impact statement

Numpydantic was developed as part of LinkML arrays (Moxon et al., 2026). Briefly, LinkML is an accessible, expressive schema language that allows a schema defined once to be translated into a number of schema and programming languages like OWL, JSON Schema, SQL DDL, etc. LinkML is widely used for linked data schemas and ontologies, and is increasingly an adoption target for scientific data formats. Support for n-dimensional arrays is a critical part of using LinkML for scientific data, and numpydantic is the core array implementation for LinkML in Python.

Numpydantic was also developed as part of an effort to rearchitect Neurodata Without Borders (NWB) (Rübel et al., 2022), the de-facto data standard for systems neuroscience (`nwb-linkml`). NWB uses HDMF (Tritt et al., 2019) as its schema language, which was developed out of necessity as most schema languages do not support arrays. HDMF and NWB are both tightly coupled to HDF5, which makes indexing and partial use of datasets extraordinarily costly - to read a scalar metadata attribute, one needs to download the entire dataset, which for neuroscientific data often means hundreds of gigabytes[6]. Backing NWB with linkml arrays and numpydantic allows array backends to be freely swapped, and separates metadata from heavier raw array data. By decoupling schema, programmatic APIs, and serialization formats with LinkML as a *lingua franca,* we are working towards a true ecosystem of FAIR data (Wilkinson et al., 2016) where data standards only need to define the schema layer and get the rest for free.

[6] See also LINDI (Neurodata Without Borders, 2024), an overlay for addressing streaming data needs with HTTP range requests.

DRAFT

Limitations & Future Directions

- The python typing spec, and standard array typing recommendations are always evolving and still in flux. Non-mypy static type checkers have trouble checking some forms of `NDArray` types. There are several intermediate solutions, like the `NDArraySchema` class, available as workarounds until it stabilizes.
- We will be extending the specification syntax to include more advanced array features, like chunking and memory order.
- Support for TypeVars that can indicate that a given axis can be any size, but all axes within a given scope must be the same size is important for many scientific applications (e.g. a timestamps array must be the same length as a video). Version 2.0.0 has experimental support for TypeVars at runtime and statically, and we will be refining this both within numpydantic itself and upstream in mypy.

AI usage disclosure

No "AI" or LLMs were used at any time during development, nor to produce any part of this submission.

Acknowledgements

Thanks to Ryan Ly, Chris Mungall, Oliver Rübel, Wouter-Michiel Vierdag, Ben Dichter, and John-Marc Chandonia for co-designing the LinkML Arrays specification which gave shape to the initial spec; to Antônio Camargo for setting up the conda forge feedstock; to all collaborators past and present; and to Rumbly Tumbly Lawnmower for being the light of my life and keeping my lap warm as I worked.

We are grateful for the funding that supported this work: NIH Director's New Innovator Award, DP2MH129986.

References

Bantilan, N. (2020). *Pandera: Statistical Data Validation of Pandas Dataframes*. 116–124. https://doi.org/10.25080/Majora-342d178e-010

Colvin, S., Jolibois, E., Ramenzani, H., Garcia Badaracco, A., Dorsey, T., Montague, D., Matveenko, S., Trylesinski, M., Runkle, S., Hewitt, D., Hall, A., & Plot, V. (2026). *Pydantic Validation* (Version 2.14.0). https://pydantic.dev/docs/validation/latest/get-started/

Curry, C. (2020). *Beartype*. beartype. https://github.com/beartype/beartype

Dask Development Team. (2016). *Dask: Library for dynamic task scheduling*. https://docs.dask.org

Grönholm, A. (2015). *Typeguard*. https://github.com/agronholm/typeguard

Hagenaars, R. (2019). *Nptyping*. https://github.com/ramonhagenaars/nptyping

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Kidger, P. (2022). *Jaxtyping*. https://github.com/patrick-kidger/jaxtyping

DRAFT

Mendoza, M., Pradeep Kumar Srinivasan, Rahtz, M., & Siles, V. (2020, September 16). *PEP 646 – Variadic Generics*. Python Enhancement Proposals; Python Software Foundation. https://peps.python.org/pep-0646/

Miles, A., Kirkham, J., Durant, M., Bourbeau, J., Onalan, T., Hamman, J., Zain Patel, Shikharsg, Rocklin, M., Dussin, R., Schut, V., Andrade, E. S. D., Abernathey, R., Noyes, C., Sbalmer, Pyup.lo Bot, Tran, T., Saalfeld, S., Swaney, J., … Banihirwe, A. (2020). *Zarr-developers/zarr-python: V2.4.0* (Version v2.4.0). Zenodo. https://doi.org/10.5281/ZENODO.3773450

Moxon, S. A. T., Solbrig, H., Harris, N. L., Kalita, P., Miller, M. A., Patil, S., Schaper, K., Bizon, C., Caufield, J. H., Cuesta, S. C., Cox, C., Dekervel, F., Dooley, D. M., Duncan, W. D., Fliss, T., Gehrke, S., Graefe, A. S. L., Hegde, H., Ireland, A. J., … Mungall, C. J. (2026). LinkML: An open data modeling framework. *GigaScience*, *15*, giaf152. https://doi.org/10.1093/gigascience/giaf152

Neurodata Without Borders. (2024). *Lindi*. Neurodata Without Borders. https://github.com/NeurodataWithoutBorders/lindi

numpy. (2017). *Numpy-stubs*. https://github.com/numpy/numpy-stubs

numpy. (2025). *Numtype*. NumPy. https://github.com/numpy/numtype

NumPy 1.20.0 Release Notes. (2021, January 30). NumPy Manual. https://numpy.org/devdocs/release/1.20.0-notes.html

Python Software Foundation. (n.d.). *Typing PEPs*. Python Enhancement Proposals (PEPs). Retrieved June 10, 2026, from https://peps.python.org/topic/typing/

Rübel, O., Tritt, A., Ly, R., Dichter, B. K., Ghosh, S., Niu, L., Baker, P., Soltesz, I., Ng, L., Svoboda, K., Frank, L., & Bouchard, K. E. (2022). The Neurodata Without Borders ecosystem for neurophysiological data science. *eLife*, *11*, e78362. https://doi.org/10.7554/eLife.78362

Smith, E. V. (2017, June 2). *PEP 557 – Data Classes*. Python Enhancement Proposals (PEPs). https://peps.python.org/pep-0557/

Tartanoglu, C. H., & Heindl, C. (2022). *Pydantic-numpy*. https://github.com/caniko/pydantic-numpy/

Tritt, A. J., Rübel, O., Dichter, B., Ly, R., Kang, D., Chang, E. F., Frank, L. M., & Bouchard, K. (2019). HDMF: Hierarchical Data Modeling Framework for Modern Science Data Standards. *2019 IEEE International Conference on Big Data (Big Data)*, 165–179. https://doi.org/10.1109/BigData47090.2019.9005648

van Rossum, G., Lehtosalo, J., & Langa, Ł. (2014, September 29). *PEP 484 – Type Hints*. Python Enhancement Proposals (PEPs). https://peps.python.org/pep-0484/

Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., … Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, *3*(1), 160018. https://doi.org/10.1038/sdata.2016.18

DRAFT